MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Single-GPU mixture-of-experts (MoE) inference can stall on HDD reads when expert weights exceed the HBM and host memory assigned to a task. Managing HBM and host residency independently can discard experts that later tokens reuse. We present MECC (MoE Expert Cache Compiler), which couples profile-guided initial placement with joint HBM/CPU retention and dependency-aware weight delivery. The compiler assigns per-layer capacities; the runtime updates resident experts from unchanged router selections. In single-run FP16 Mixtral-8x7B measurements on one A100, MECC with uniform initialization delivers decode speedups of 1.77× over local streaming and 2.95× over fixed-CPU retention, with matching sample perplexity.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models increase parameter capacity while activating only a few experts per token [1]–[4]. Deployment must still keep every expert addressable because a later token may select a different subset. This paper studies single-GPU inference when the expert set exceeds HBM and cannot be retained entirely in the host memory assigned to a task. The remaining weights reside on HDD. A selected expert must then be read into host memory and copied to the GPU before execution. The problem is to reduce these deliveries under fixed fast-tier allowances while preserving the model's original routing and weights. This setting matters for local inference and shared servers, where one task may receive only part of the machine's physical memory.

Existing systems address several parts of this problem. Multi-GPU expert parallelism distributes resident weights across accelerators [5]–[7]. Single-GPU offloading systems exploit activation traces, CPU execution, prefetching, or transfer pipelines [8]–[12]; SSD-LLaMA and edge FlashMoE further optimize disk delivery and replacement [13], [14]. These mechanisms make initial placement, admission, and retention consequential choices. A remaining question is how a profile-derived initial state and online replacement should coordinate HBM and host residency when some experts must stay outside both tiers. Figure 1 isolates the runtime aspect with local controls on one A100. Separate HBM/CPU caches take 32.94 s per decode token, versus 7.38 s with joint retention under the same nominal slots and initialization. Their reported HDD-demand shares are 47.9% and 12.5%, respectively. These observations motivate preserving reuse across the tier boundary; they do not constitute a reproduction of the cited systems.

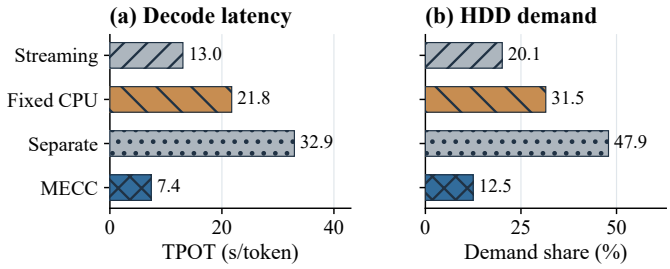


Fig. 1. Local retention controls on one A100 with Mixtral-8x7B. Separate denotes independently managed HBM/CPU caches. All policies use the same nominal slots and eight-prompt input. HDD-demand share includes all instrumented phases. One run per policy.

We present MECC, the MoE Expert Cache Compiler, to connect initial placement with runtime retention. An independent route profile supplies an expert-demand prior. The compiler ranks experts within each layer and allocates HBM and CPU slots under the task's allowances. At runtime, the unchanged router provides actual demand. A CPU hit promotes the selected expert and retains the displaced HBM expert in CPU memory, preserving it for later tokens. Online recency changes the resident identities as the workload changes, while the compiled capacities remain fixed. Weight delivery orders reads, copies, and GPU consumers through readiness dependencies. These decisions address different parts of the same lifecycle: the profile selects an initial state, joint retention preserves reuse, and the delivery contract makes requested weights available before execution. The current A100 evaluation uses uniform initialization to examine runtime behavior independently of a profile-placement gain.

This paper makes three contributions. **(1) Placement compilation:** we define a per-layer interface that maps route profiles and task allowances to ranked expert IDs and initial tier capacities, together with a marginal-coverage allocation heuristic. **(2) Joint retention and delivery:** we specify promotion, victim demotion, and readiness dependencies across HBM, CPU pinned memory, and HDD while retaining the router and FP16 weights. **(3) Single-GPU evaluation:** we implement an end-to-end harness and compare the runtime with streaming, fixed-CPU retention, independent caches, and local alternative policies on Mixtral-8x7B. With uniform initialization, MECC takes 7.38 s per output token, compared with 13.04 s for streaming and 21.75 s for fixed-CPU retention; the latter two controls report the same sample perplexity as MECC. The evaluation also examines source-tier demand and host-slot sensitivity.

II. Background and Related Work

A. Sparse Execution and Deployment Choices

For layer ℓ, a router selects k_ℓ of N_ℓ experts for each token. Only those experts execute, but any of the N_ℓ may be selected by a later token. Switch Transformer and GShard establish the sparse-routing model [2], [15]; Mixtral, DeepSeekMoE, and DeepSeek-V2/V3 show large language-model instances [3], [4], [16], [17]. The active parameter count does not by itself specify the required storage capacity.

With enough aggregate GPU memory, expert parallelism places weights on several GPUs and routes token activations to them. DeepSpeed-MoE, Tutel, and MegaBlocks develop execution and scheduling mechanisms for distributed MoE workloads [5], [6], [18]. Distributed FlashMoE fuses dispatch, compute, and combine into a persistent kernel [7]; HarMoEny balances token work across GPUs [19]. These are relevant deployment alternatives, but their published multi-GPU operator numbers are not single-A100 baselines. Mixtral 8x22B provides a second example of how expert capacity can grow beyond a single card [20].

For one GPU, HBM must also hold dense layers, activations, and KV cache. PagedAttention and SGLang address other parts of the serving-memory and execution problem [21], [22]. CPU memory can retain additional experts; a backing store retains the rest. A miss moves weights from the selected source before the expert kernel executes. FlexGen, DeepSpeed Inference, ZeRO-Infinity, and LLM in a Flash establish related heterogeneous-memory techniques [23]–[26]. PowerInfer exploits activation skew for CPU/GPU inference [27]. Their placement and scheduling goals motivate explicit accounting of both resident and temporary copies.

B. Related Expert-Serving Systems

SwapMoE exposes a tunable memory budget; Fiddler compares CPU expert execution with weight movement; MoE-Lightning pipelines CPU, GPU, and I/O work; and MoE-Infinity uses request-level activation traces [9]–[11], [28]. Klotski pipelines expert-aware batches [29]. ExpertFlow predicts routes and schedules tokens, while ProMoE, Pre-gated MoE, and Fate prefetch experts from intermediate signals [12], [30]–[32]. DAOP combines data-aware CPU/GPU execution with predictive pre-calculation [33]. Those predictors change the information available before a gate. Our demand-only retention rule uses the unchanged gate output.

HOBBIT uses mixed-precision expert loading, and Specter pairs quantization with speculative offloading [34], [35]. SSD-LLaMA coordinates disk I/O, tiered residency, and execution placement [13]. The edge-device FlashMoE learns a recency/frequency replacement rule [14]; MoE-CORE combines domain initialization, layer-specific capacities, and prefetch [36]. ZipMoE adds lossless compression and cache-affinity scheduling [37]. These systems overlap individual MECC design axes. Our local RF adapter is not a reproduction of the published FlashMoE system.

MoEpic splits experts into segments and chooses cache allocations; SliceMoE caches precision slices [38], [39]. Mira couples predictive staging with quantized transfers [40]. OD-MoE uses on-demand loading across edge nodes; Bevy interleaves concurrent requests during expert fetches [41], [42]. HeteGen studies heterogeneous CPU/GPU inference for dense LLMs [43]. These systems change granularity, prediction, precision, or request scheduling; MECC's measured comparison is limited to the local policies defined in Section VI.

The comparison spans three decisions. *Execution location* determines whether to move weights to the GPU or execute an expert on the CPU. Fiddler and DAOP explore this choice [10], [33], whereas the measured MECC path executes selected experts on the GPU. *Lookahead* determines when an expert becomes known. ProMoE, Pre-gated MoE, Fate, and Mira expose or predict later demand [12], [31], [32], [40]; MECC's demand path begins with the original gate output. *Residency* determines which delivered weights survive for reuse. MoE-Infinity, FlashMoE, MoE-CORE, and ZipMoE address this decision with different trace, replacement, initialization, or scheduling mechanisms [9], [14], [36], [37]. These axes can coexist in one system, so the experimental controls must identify the particular rule being changed.

The storage regime also affects the comparison. A host-resident expert set mainly exposes PCIe movement or CPU execution. A set larger than the task's fast-tier allowance additionally exposes backing-store reads. SSD-LLaMA and edge FlashMoE explicitly address storage delivery [13], [14]. MECC studies HDD-backed demand while combining a placement prior with online HBM/CPU retention.

III. Motivation and Insights

A. Initial Coverage across Layers

Figure 2(a) illustrates initial coverage. The expert set is larger than the fast-tier slots assigned to a task. Equal slots per layer can waste capacity when layers receive different fractions of demand. The compiler can rank experts from an independent profile and allocate the next slot where it covers the most expected selections. This is an initial-state decision; it does not change router outputs. Routing locality differs across models and workloads [44]. The current A100 main runs use uniform slots and natural expert order, so their speedup does not measure this insight's profile-guided contribution.

B. Preserving Reuse after Eviction

Figure 2(b) illustrates reuse across the tier boundary. An HBM victim need not become cold immediately. A later token may select the same expert. Separate HBM and CPU replacement rules can leave a duplicate resident or evict a useful host copy. In a matched local control, independent caches report 47.9% cold-tier demand and 32.94 s/token; fixed CPU retention reports 31.5% and 21.75 s/token; joint retention reports 12.5% and 7.38 s/token. The insight is to treat HBM and host residency as one logical reuse stack: promotion makes room for the HBM victim in host memory. The comparison motivates the runtime rule under the tested quotas; physical bytes and phase-specific I/O remain separate accounting questions.

C. Correcting a Stale Initial State

Figure 2(c) shows the timing of adaptation. A profile describes a prior workload, not every later request. When the selected expert distribution changes, static residency can keep experts that the new workload rarely uses. The compiler therefore fixes initial capacities and ranked IDs, while online recency changes the resident identities. This division gives the runtime a way to correct a stale prior without modifying the router. A task switch requires two distinct input sets and a measured before/after trace. The present single-A100 main run does not test a switch.

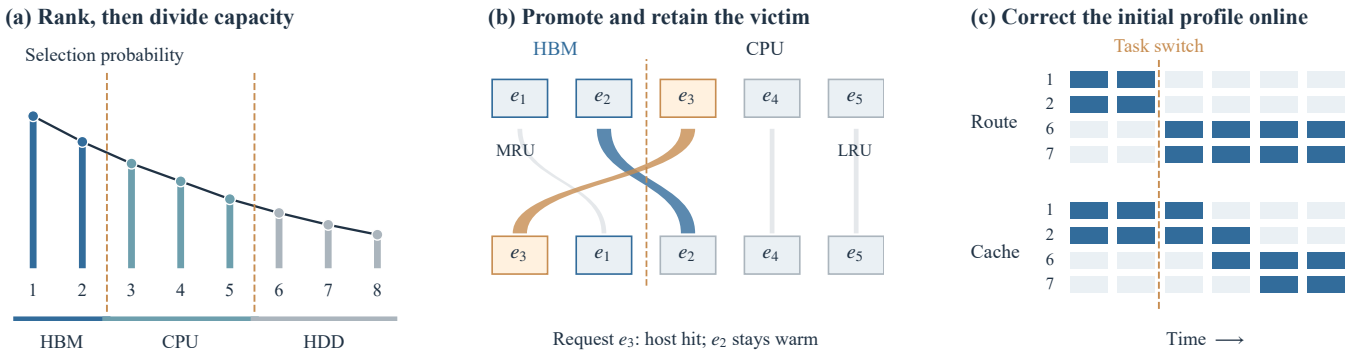


Fig. 2. Profile-guided placement and online correction. (a) Rank experts within a layer and divide fast-tier capacity at the ranked boundaries. (b) Treat HBM and CPU as one reuse stack: requesting e_3 promotes it while retaining the HBM victim e_2 in CPU. (c) Update resident identities when the routed expert set changes. All probabilities, states, and time windows are schematic.

IV. Overview

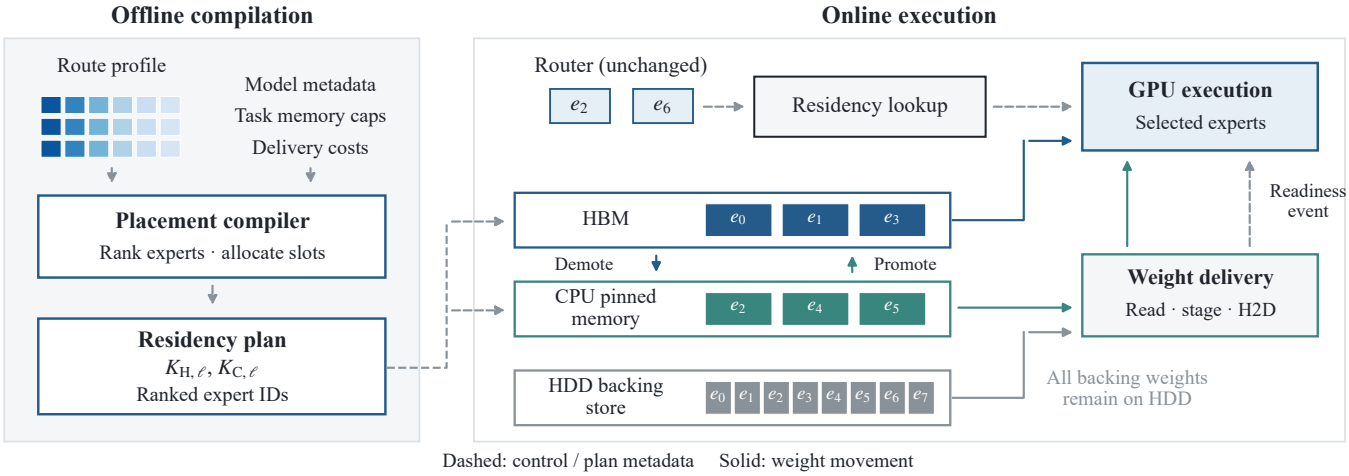


Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution resolves unchanged router selections against HBM, CPU pinned memory, and HDD, then delivers weights before their GPU consumers. Dashed arrows carry control or plan metadata; solid arrows move weights.

Figure 3 separates offline compilation from online execution. An independent route profile, model metadata, task memory caps, and delivery costs define the compilation inputs. The output specifies layer capacities and initial expert IDs. Mirrors and staging are charged outside unique primary residency; HDD retains all backing weights.

The unchanged router supplies demand IDs. Residency lookup selects a source tier; the runtime promotes requested experts and demotes HBM victims. The delivery path enforces weight readiness before execution. Layer capacities stay fixed during a run, while expert identities and recency change. Planner controls therefore fix online rules; replacement controls fix quotas and initialization (Section VI-C).

V. Method

A. Compiler: Profile-Guided Placement

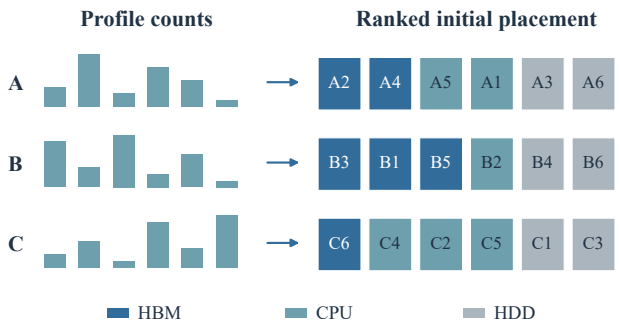


Fig. 4. Compiling layer-specific placement. A, B, and C denote different layers. The compiler keeps each layer's expert identities, ranks its profile counts, and allocates HBM and CPU slots from shared task allowances. Colors identify initial tiers; counts and capacities are schematic.

Figure 4 shows the compiler's input and emitted state. The compiler takes model metadata, memory budgets, delivery costs, and an independent route profile. The profile supplies request probability p_e for each expert. For equal-size experts within a layer, a discrete Zipf model summarizes the ranked probabilities:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}, \qquad H_n(\alpha) = \sum_{r=1}^{n} r^{-\alpha}. \tag{1}$$

Here $q_{\ell,r}$ is conditional on an expert selection in layer ℓ. Let w_ℓ be that layer's fraction of all selections. The global probability is $p_e = w_\ell q_{\ell,r}$, with $\sum_\ell w_\ell = 1$. For fixed top-k routing, $w_\ell = k_\ell / \sum_j k_j$. Each rank retains its profiled expert ID. Empirical probabilities can replace the fitted Zipf distribution. Neither representation encodes request order.

Let $x_{e,t} \in \{0, 1\}$ indicate the primary tier of expert e. The tiers are HBM, CPU, and HDD, denoted by H, C, and S. Residency budgets $\bar{B}_t$ must exclude buffers and mirrors. The following static objective is a reference model, not a claim that the current planner solves the full problem:

$$\begin{aligned} \min_x \quad & \sum_e p_e \sum_t x_{e,t} T_t(s_e), \\ \text{s.t.} \quad & \sum_t x_{e,t} = 1, \quad \sum_e s_e x_{e,t} \le \bar{B}_t \quad (t \in \{\mathrm{H}, \mathrm{C}\}). \end{aligned} \tag{2}$$

The affine model $T_t(s) = a_t + s/b_t$ represents fixed overhead and effective bandwidth. Calibration uses the target machine. Equation 2 estimates static delivery cost under the profile. Runtime locality, migration, and overlap are evaluated separately.

Proposition 1: Assume equal-size objects, fixed request probabilities, independent tier capacities, and $T_\mathrm{H} \le T_\mathrm{C} \le T_\mathrm{S}$. An optimal static placement fills HBM with the most probable objects, CPU with the next most probable objects, and HDD with the remainder.

Proof sketch. Suppose $p_i > p_j$ and object i occupies the slower tier. Exchanging the two objects changes expected cost by $(p_i - p_j)(T_\mathrm{fast} - T_\mathrm{slow}) \le 0$. Repeated exchanges produce ranked placement. Filling an empty faster-tier slot also cannot increase cost. The result applies to the stated static model.

The multilayer implementation allocates slots by marginal coverage. Under the Zipf model, a slot covering rank r adds

$$\Delta_\ell(r) = w_\ell q_{\ell,r}. \tag{3}$$

This is a coverage score, not the exact latency marginal of Equation 2. Tier-boundary movements can affect both HBM and CPU assignments. The heuristic allocates HBM slots first, then CPU slots from the remaining ranks. At each step it selects the largest next-rank score; ties use layer ID. Expert-ranking ties use expert ID. It stops at the tier's slot cap or exhaustion of experts. Per-layer minima, if enabled, are assigned first and an infeasible minimum allocation must be rejected. A direct scan costs $O(L(S_\mathrm{H} + S_\mathrm{C}))$ after ranking, where S_t is a total slot cap. These rules define the compiler interface. Batched prefill additionally requires distinguishing token selections from distinct expert fetches. The sequential heuristic does not solve the general static objective.

The plan records per-layer capacities, ranked expert IDs, initial tiers, and modeled hit fractions. A missing profile selects natural expert order. The measured runs below use this uniform/natural-order path, so they test the runtime retention and delivery stack rather than the profile compiler. Profile initialization cost and gains require separate matched runs.

B. Runtime: Cross-Tier Retention

Each layer has ordered HBM and CPU residency maps. An HBM hit updates recency. A CPU hit removes the expert from CPU residency and promotes it to HBM as most recently used. An HDD miss reads the weights into staging before H2D transfer. When HBM is full, its least-recently used expert is demoted to CPU. CPU pressure evicts the least-recently used host resident. The HDD backing weights remain available.

Figure 5 starts with HBM $[e_1, e_2]$ and CPU $[e_3, e_4]$, each ordered from MRU to LRU. Requesting e_3 promotes it and demotes e_2. A later request for e_2 hits CPU. This example uses exclusive logical residency. An implementation can keep a CPU mirror to avoid device-to-host (D2H) transfer on eviction. The mirror consumes host memory and is charged separately.

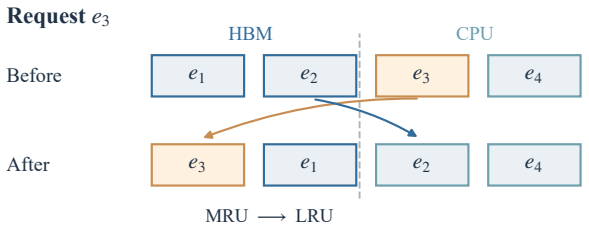


Fig. 5. One CPU-hit transition. Lists run from MRU to LRU. Requesting e_3 promotes it, demotes the HBM victim e_2, and preserves both experts in the combined fast-tier set. The dashed line is the capacity boundary.

For a serial demand stream within one layer, concatenate the HBM MRU-to-LRU stack with the CPU MRU-to-LRU stack. Exclusive promotion and victim demotion preserve the corresponding LRU stack of capacity $K_\mathrm{H} + K_\mathrm{C}$. This reference requires matching initialization, no prefetch or alternative admission, and completed transitions before the next request. It does not imply equivalence to a cache that reallocates capacity across layers, or dominance over static placement.

C. Delivery: Dependency-Aware Overlap

A correct asynchronous implementation requires a readiness event after each pinned-host copy. The computation stream must wait before consuming the weight. The source buffer must remain live until copy completion, and the resident weight until its last consumer finishes. These are required lifetime rules, including across separate loading and dispatch iterators; the execution stream must enforce them with readiness events.

At batch one, routing reveals the selected IDs only after the current layer's gate. Copying one selected expert may overlap computation of another already-ready expert if kernel and aggregation order permit. This provides no knowledge of a later layer's route. In-flight weights must be protected from eviction. Replacement comparisons must disable predictive prefetch; the overlap control must hold admission, residency, and requests fixed. The available overlap and its exposed critical path remain measurements, rather than an assumed gain from a copy stream.

Figure 6 pairs HBM-resident e_1 with HDD-backed e_3 from the same gate. HDD read precedes H2D and readiness. Copy completion releases the pinned source; final-consumer completion permits GPU eviction. Useful independent computation determines the available overlap.

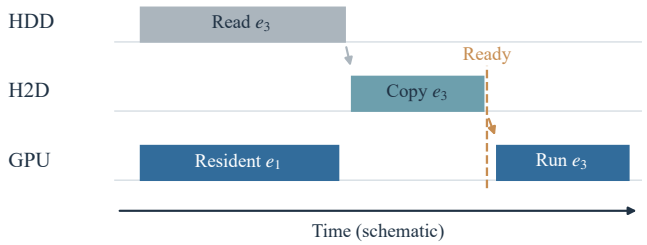


Fig. 6. Delivery dependencies within one gate's selected set. The GPU can execute resident e_1 while e_3 is read and copied. A readiness event orders the copied weight before its GPU consumer. Durations are schematic.

The design requires unchanged weights, selected experts, kernels, and aggregation order. Under these conditions, changing residency preserves the forward computation. The implementation also needs correct synchronization. The validation protocol compares synchronous and asynchronous paths on identical inputs using weight hashes, route selections, logits, tokens, and perplexity; the same protocol isolates transfer scheduling from changes in numerical execution.

D. Implementation and Measurement Contract

Model adapters map layer/expert IDs to immutable FP16 weights. Policies share the loader and router interface. Memory accounting includes physical mirrors and staging buffers.

Physical accounting deduplicates storage identities, not expert IDs: distinct live copies still consume bytes. Let R_t be resident expert bytes, D_{H} distinct host mirrors, S_t staging, and O_t other charged allocations. The required constraints are

$$R_{\mathrm{C}} + D_{\mathrm{H}} + S_{\mathrm{C}} + O_{\mathrm{C}} \le B_{\mathrm{C}}, \tag{4}$$

$$R_{\mathrm{H}} + W_{\mathrm{dense}} + W_{\mathrm{KV/act}} + S_{\mathrm{H}} + O_{\mathrm{H}} \le B_{\mathrm{GPU}}. \tag{5}$$

Full mirroring requires $D_{\mathrm{H}} \ge R_{\mathrm{H}}$. Residency must be reduced or a mirror-free path validated. Peak measurements must include initialization and transient copies, not only steady residency.

Counters must separate initialization, warmup, prefill, decode, and quality evaluation. TPOT divides decode time by measured intervals; TTFT includes prefill and the first output. Demand-miss share is the fraction of selected-expert arrivals without ready HBM/CPU weights. In-flight prefetch is logged separately. Physical HDD/H2D/D2H bytes use the same phase boundaries. These counters define the same phase boundaries for every policy.

VI. Evaluation

A. Testbed and Comparison

The model is Mixtral-8x7B-Instruct-v0.1 with FP16 expert weights on one A100 80 GB PCIe GPU. The backing store resides on a RAID logical block device reported as rotational (`ROTA=1`); the physical drives behind the controller have not been inventoried. We call the cold tier HDD as a deployment shorthand. No SSD/NVMe comparison is measured.

The evaluation asks three questions: how much does joint retention change end-to-end latency; where are selected experts found; and how does the host-slot allowance affect both? The counters include warmup and perplexity as well as generation. We therefore report cold-tier demand share as a cache observation, not as a decode-only explanatory variable. TPOT measures the decode interval, whereas TTFT includes the first response.

Each main policy has one completed end-to-end run on the same model revision with nominal expert allowances of 50 GB HBM and 40 GB host memory, four HBM and three host slots per layer, uniform initialization, and cold reads enabled. The input source is WikiText-2 test: eight prompts truncated to 64 input tokens, with 64 generated tokens each. Perplexity uses a 1024-token WikiText-2 sample. Values are single-run point estimates. The nominal host allowance is a planner parameter; mirrors, staging, and page cache are not demonstrated to fit within a physical 40 GB process cap.

The compared policies are unoptimized streaming (`naive`), fixed-CPU retention (`swapmoe`), MECC, a local recency/frequency adapter, a CPU-compute adapter, and MECC with an alternative I/O path (`ours_io`). The RF and I/O adapters are local implementations, not reproductions of published FlashMoE or SSD-LLaMA. These six runs use one A100 each. There is no measured multi-GPU reference.

B. Single-GPU Latency

Figure 7 reports TPOT of 7.38 s for MECC, 13.04 s for streaming, and 21.75 s for fixed-CPU retention. The corresponding TPOT ratios are 1.77× and 2.95×. The local RF adapter is slightly faster at 7.19 s; MECC I/O takes 7.78 s and the CPU-compute adapter takes 8.26 s. The cold-tier demand shares are 12.5% for MECC, 20.1% for streaming, 31.5% for fixed CPU, and 11.1% for RF. These multi-phase counts describe cache behavior, rather than decode-only I/O. The RF adapter has the lowest TTFT and TPOT in this local comparison.

The first-token times are 139.1 s for MECC, 156.9 s for streaming, 174.2 s for fixed CPU, 101.8 s for RF, and 398.4 s for CPU compute. MECC, streaming, fixed CPU, RF, and MECC I/O report perplexity 3.427 on the same 1024-token sample; CPU compute reports 3.432. This quality comparison is specific to that sample. A separate short-window route/logit/token comparison supports the implementation check but is not an independent full-resident reference.

C. Retention Control

The independent-cache run matches the main runs' model, nominal slots, input scale, and uniform initialization. Figure 8 compares independent caching, fixed CPU, and joint MECC retention. Independent caching takes 32.94 s/token and reports 47.9% cold-tier demand; the fixed-CPU policy takes 21.75 s and reports 31.5%; MECC takes 7.38 s and reports 12.5%. The ordering is consistent in both latency and demand sources. Independent caching reaches HBM for 51.9% of selected experts, fixed CPU reaches HBM for 62.6%, and MECC reaches HBM for 77.5%. The CPU shares are 0.3%, 5.8%, and 10.0%, respectively. This comparison supports the joint-retention mechanism under these local policies. Physical host footprints and exposed I/O time still need matched accounting before the entire latency gap can be assigned to replacement alone.

D. Host-Slot Sensitivity

The 30 and 60 GB nominal CPU settings use the same four-prompt, 16-token WikiText-2 protocol within each setting. They are a distinct experiment from the eight-prompt main comparison. At 30 GB, MECC takes 14.46 s/token and reports 32.2% cold-tier demand; at 60 GB, it takes 0.76 s/token and reports no cold-tier demand. The 60 GB plan has four HBM and four host slots per layer, enough for all eight experts. The RF adapter takes 13.06 s/token at 30 GB and 0.74 s/token at 60 GB, faster than MECC in both settings. Figure 9 shows all five local policies at these two settings. The sharp 60 GB drop is a capacity transition: the nominal HBM and host slots together cover all eight experts per layer, so the cold tier is no longer exercised by demand. It should not be interpreted as an HDD optimization gain at that budget.

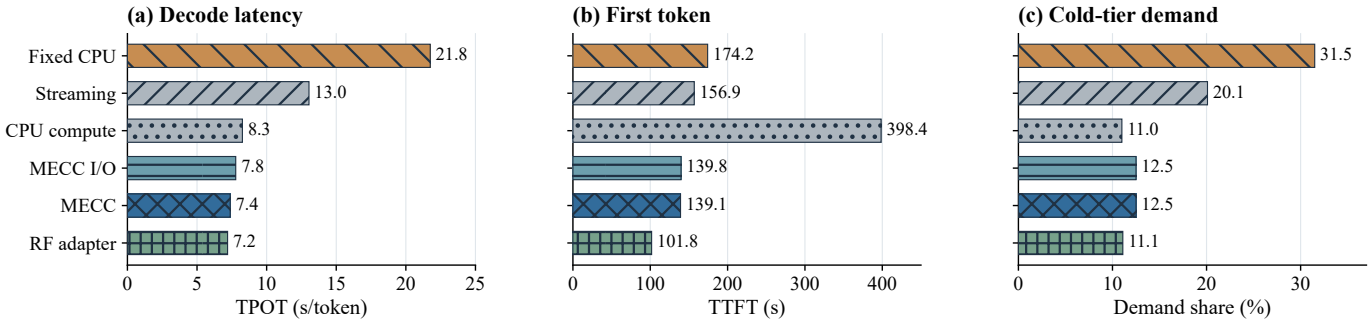


Fig. 7. Single-A100 Mixtral-8x7B comparison (eight WikiText-2 prompts, 64 output tokens, one run per policy). (a) Decode time per output token. (b) First-token latency. (c) Reported cold-tier demand share across all instrumented phases, including warmup and perplexity; it is not a decode-only share. All six local policies are shown.

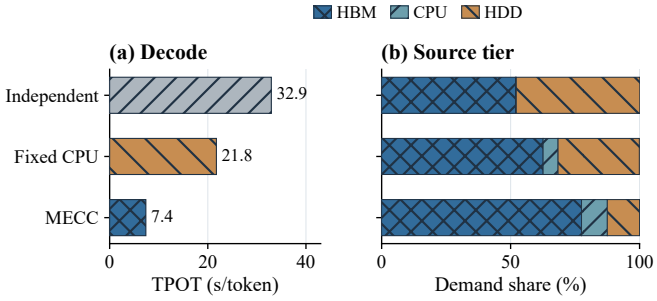


Fig. 8. Matched single-A100 cache-policy control at 50/40 GB nominal allowances, eight WikiText-2 prompts, and 64 output tokens. (a) Decode latency. (b) Selected-expert demand resolved from HBM, CPU, or HDD across all instrumented phases. Independent and fixed-CPU policies are local mechanisms. One run per policy.

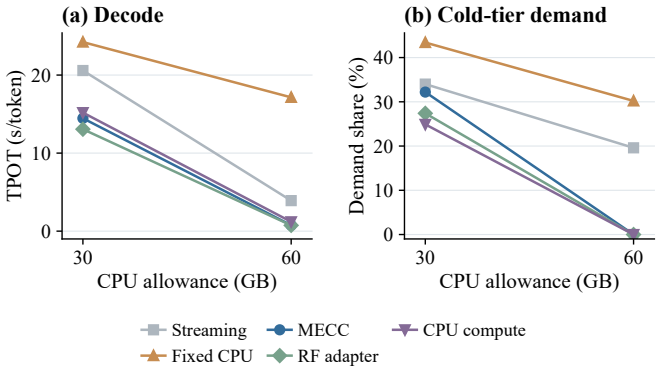


Fig. 9. Nominal CPU-slot sensitivity on one A100. Four WikiText-2 prompts and 16 output tokens per run; 30 and 60 GB are the only two settings shown with the same input source. The earlier 20 GB runs used a different prompt source and are excluded from this comparison. Cold-tier share covers all instrumented phases. One run per point.

E. Reading the Capacity Transition

The 60 GB plan has four HBM and four CPU slots per layer, covering all eight experts. MECC's zero cold-tier demand is consistent with this slot geometry. It does not mean that the model fits in HBM. At the smaller quota, the cache must decide which experts remain outside the fast tiers, and the replacement rule changes the observed reuse. The main and budget protocols use different prompt/output lengths, so only points within the matched budget protocol belong on the same curve.

F. Scope of the Measurements

These results are single-run observations on a RAID logical device reporting rotational media. The main comparison uses uniform initialization and does not measure the compiler's profile-guided gain. A true WikiText-2 to GSM8K task-switch sequence has now been implemented, but no completed task-switch result appears in the committed result set. The 20 GB budget runs mix prompt sources and cannot be joined to the 30/60 GB curve. Physical drive identity, physical host-cap equivalence, and multi-GPU resident performance remain separate measurements.

VII. Conclusion

On one A100 with uniform initial placement, MECC's joint-retention runtime reduces decode latency relative to local streaming, fixed-CPU, and independent-cache controls. The local RF adapter is slightly faster on the main workload. Profile-guided placement, task changes, physical budget equivalence, and multi-GPU tradeoffs require separate evidence before they enter the performance claim.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[10] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[11] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[12] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[13] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[14] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[15] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[16] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[17] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[18] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[19] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[20] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[21] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[22] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[23] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[24] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[25] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[26] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[27] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[28] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[29] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[30] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[31] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[32] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[33] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[34] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[35] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[36] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[37] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[38] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[39] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[40] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[41] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

[42] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[43] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.

[44] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.